

DOMAĆI ZADATAK

PROBLEM TRGOVAČKOG PUTNIKA

Travelling Salesman Problem (TSP)

Pohlepna pretraga sa dve heurističke funkcije
Implementacija u Python 3 i MATLAB/Octave
Analiza rezultata i poređenje sa egzaktnim rešenjem

STUDENT: Anita Mijatović
BROJ INDEKSA: 612/2022
PREDMET: Veštačka inteligencija (IV godina, VIII semestar)
DATUM: Kragujevac, maj 2026. godine

Sadržaj izveštaja

1.	Opis zadatka i definicija problema	3
1.1	Matrica rastojanja i analiza grafa	3
2.	Teorijska osnova	4
2.1	Formalizacija TSP-a i prostor stanja	4
2.2	Pohlepna pretraga — karakteristike	4
3.	Trag izvršavanja H1: Nearest Neighbor	5
4.	Trag izvršavanja H2: Greedy Edge Insertion	6
5.	Referentno optimalno rešenje (Brute Force)	7
6.	Poređenje i analiza rezultata	8
7.	Implementacija u Python 3	9
8.	Implementacija u MATLAB / Octave	11
9.	Zaključak i teorijska razmatranja	13

1. Opis zadatka i definicija problema

Ovaj domaći zadatak bavi se rešavanjem **Problema Trgovačkog Putnika** (TSP — *Travelling Salesman Problem*) na grafu od 5 gradova: **A, B, C, D** i **E**. Problem je jedan od najpoznatijih NP-teških problema kombinatorne optimizacije i od izuzetnog je praktičnog i teorijskog značaja u oblasti veštačke inteligencije.

Cilj je, polazeći iz čvora A, pronaći minimalan Hamiltonov ciklus koji:

- Posećuje svaki od gradova (B, C, D, E) **tačno jednom**,
- Vraća putnika nazad u polazni grad A,
- Minimizuje ukupnu dužinu (zbir težina) pređenih ivica.

Za rešavanje se primenjuju dve heurističke funkcije u okviru **pohlepne pretrage** (Greedy Best-First Search), a rezultati se porede sa egzaktnim rešenjem (Brute Force). Implementacija je urađena u **Python 3** i **MATLAB/GNU Octave**.

1.1 Matrica rastojanja i analiza grafa

Ulazni graf ima 5 čvorova i definisan je sledećom matricom rastojanja (vrednost ∞ označava da direktna veza ne postoji):

	A	B	C	D	E
A	∞	7	6	10	13
B	7	∞	7	∞	10
C	6	7	∞	8	9
D	10	∞	8	∞	6
E	13	10	9	6	∞

Tabela 1.1: Matrica rastojanja između 5 gradova (∞ = nema direktne veze)

Napomena: Graf nije kompletan — ne postoji direktna veza između čvorova **B** i **D**. Ova specifičnost direktno utiče na pohlepne odluke algoritama.

2. Teorijska osnova

2.1 Formalizacija TSP-a i prostor stanja

TSP spada u klasu **NP-teških** kombinatornih optimizacionih problema. Za n gradova, broj jedinstvenih Hamiltonovih ciklusa iznosi $(n-1)! / 2$. Za 5 gradova: $(5-1)!/2 = 12$ **jedinstvenih ruta**. Egzaktno rešenje Brute Force metodom je izvodljivo za mali n , ali pošto složenost raste faktorijski, za $n > 20$ postaje računski neizvodljivo.

Komponenta	Opis
Stanje	(trenutni grad, skup posećenih gradova)
Početno stanje	(A, {A})
Ciljno stanje	(A, {A,B,C,D,E}) — svi gradovi posećeni, povratak u A
Operator	Prelaz iz trenutnog u neposećen grad sa direktnom ivicom
Cena puta	Suma rastojanja svih pređenih ivica u ciklusu

Tabela 2.1: Formalizacija prostora stanja TSP problema

2.2 Pohlepna pretraga — karakteristike i ograničenja

Pohlepna pretraga (*Greedy Best-First Search*) u svakom koraku bira lokalno optimalan potez isključivo na osnovu heurističke funkcije $h(n)$, ignoriše cenu pređenog puta $g(n)$. Formula: $f(n) = h(n)$.

Osobina	Pohlepna pretraga	A* algoritam
Ocenska funkcija	$f(n) = h(n)$	$f(n) = g(n) + h(n)$
Kompletnost	NIJE garantovana	Garantovana (adm. h)
Optimalnost	NIJE garantovana	Garantovana (adm. h)
Prostorna složenost	$O(b^m)$	$O(b^d)$
Brzina (praksa)	Brza, ali rizična	Sporija, ali pouzdana

Tabela 2.2: Poređenje pohlepne pretrage i A* algoritma

3. Trag izvršavanja H1: Nearest Neighbor heuristika

Definicija i algoritam

H1 procenjuje vrednost čvora n na osnovu rastojanja do **najbližeg susednog neposećenog grada**:

$$h1(n) = \min \{ \text{dist}(n, v) \mid v \notin \text{posećeni} \}$$

Algoritam polazi iz $\text{start}=A$, $\text{posećeni}=\{A\}$. U svakoj iteraciji bira susedni neposećeni čvor sa minimalnom distancom, dok ne obiđe sve gradove i zatvori ciklus povratkom u A .

Detaljan trag izvršavanja (iz koda)

Korak	Trenutni grad	Kandidati (grad: dist)	Izabran (h min)	Deo rute
1	A	B:7, C:6, D:10, E:13	C (h=6)	$A \rightarrow C$
2	C	B:7, D:8, E:9	B (h=7)	$A \rightarrow C \rightarrow B$
3	B	D: ∞ , E:10	E (h=10)	$A \rightarrow C \rightarrow B \rightarrow E$
4	E	D:6	D (h=6)	$A \rightarrow C \rightarrow B \rightarrow E \rightarrow D$
↩	D	Povratak u A (dist=10)	—	$A \rightarrow C \rightarrow B \rightarrow E \rightarrow D \rightarrow A$

Tabela 3.1: Korak-po-korak trag izvršavanja H1 heuristike

H1 rezultat: $A \rightarrow C \rightarrow B \rightarrow E \rightarrow D \rightarrow A$ | Ukupna cena: $6 + 7 + 10 + 6 + 10 = 39$

Analiza pohlepne greške u koraku 2

Ključna slabost algoritma manifestuje se u **koraku 2**. Iz čvora C, najbliži neposećeni grad je B ($d=7$), a D je za jednu jedinicu dalji ($d=8$). Odabirom B primorao je algoritam da se na kraju nađe u D sa jedinom opcijom $D \rightarrow A$ ($d=10$). Da je izabran D, ruta bi bila: **$A \rightarrow C \rightarrow D \rightarrow E \rightarrow B \rightarrow A = 6+8+6+10+7 = 37$** — optimum. Ovo je klasičan primer "*problema poslednje milje*" kod lokalnih pohlepnih strategija.

4. Trag izvršavanja H2: Greedy Edge Insertion heuristika

Definicija i algoritam

H2 donosi odluke **globalno** — posmatra sve ivice grafa istovremeno. Vrednost svake ivice $e=(u,v)$ je njena dužina: $h2(e) = \text{dist}(u,v)$. Algoritam sortira ivice rastuće i pohlepno ih ubacuje uz dva strukturna uslova:

- **Uslov 1 (Stepen):** Nijedan čvor ne sme imati stepen veći od 2.
- **Uslov 2 (Ciklus):** Ilica ne sme zatvoriti prevremeni podciklus — proverava se **Union-Find** strukturom ($O(\alpha(n)) \approx O(1)$). Zatvaranje je dozvoljeno samo u poslednjem koraku.

Detaljan trag izvršavanja (iz koda)

Ivica	Rastojanje	Stepeni pre	Status	Obrazloženje
A – C	6	A:0, C:0	ODABRANA ✓	Stepeni OK, nema ciklusa
D – E	6	D:0, E:0	ODABRANA ✓	Stepeni OK, zasebna komponenta
A – B	7	A:1, B:0	ODABRANA ✓	Stepeni OK, spaja A komponentu
B – C	7	B:1, C:1	ODBIJENA ✗	Zatvara preuranjeni krug A-C-B-A
C – D	8	C:1, D:1	ODABRANA ✓	Spaja komponente {A,B,C} i {D,E}
C – E	9	C:2, —	ODBIJENA ✗	Čvor C već ima stepen 2
A – D	10	A:2, —	ODBIJENA ✗	Čvor A već ima stepen 2
B – E	10	B:1, E:1	ODABRANA ✓	Zatvara finalni Hamiltonov ciklus

Tabela 4.1: Korak-po-korak trag izvršavanja H2 heuristike

Odabrane ivice:	A-C (6)	D-E (6)	A-B (7)	C-D (8)	B-E (10)
-----------------	---------	---------	---------	---------	----------

H2 rezultat: A → C → D → E → B → A | Ukupna cena: 6 + 8 + 6 + 10 + 7 = 37 ★ OPTIMALNO

5. Referentno optimalno rešenje (Brute Force)

Iscrpna pretraga isprobava sve permutacije gradova. Za 5 gradova to je $(5-1)! = 24$ **permutacije** (12 jedinstvenih ciklusa bez duplikata po smeru).

Ruta	Detalji cene	Ukupno	Status
A→B→C→D→E→A	7+7+8+6+13	41	
A→B→C→E→D→A	7+7+9+6+10	39	
A→B→E→D→C→A	7+10+6+8+6	37	★ OPTIMUM
A→C→B→E→D→A	6+7+10+6+10	39	H1 rešenje
A→C→D→E→B→A	6+8+6+10+7	37	H2 rešenje = optimum
A→C→E→D→B→A	6+9+6+∞	∞	Nemoguća ruta (D→B=∞)
A→D→C→B→E→A	10+8+7+10+13	48	
A→D→E→B→C→A	10+6+10+7+6	39	

Tabela 5.1: Odabrane rute iz Brute Force pretrage (prikazano 8 od 12)

Globalni optimum: cena = 37

Optimalne rute: A→B→E→D→C→A i A→C→D→E→B→A (isti ciklus, suprotni smer).

Odsustvo ivice B-D znači da svaka validna ruta mora koristiti put kroz C ili E za prelaz između B i D dela grafa.

6. Poređenje i analiza rezultata

Algoritam	Pronađena ruta	Cena	Odstupanje	Status
Brute Force (Optimum)	A→B→E→D→C→A	37	0.0%	Egzaktni minimum
H1: Nearest Neighbor	A→C→B→E→D→A	39	+5.4%	Suboptimalno
H2: Greedy Edge Insertion	A→C→D→E→B→A	37	0.0%	★ Optimalno!

Tabela 6.1: Zbirna tabela performansi svih triju algoritama

6.1 Analiza ponašanja H1 (Nearest Neighbor)

Algoritam H1 je pokazao slabost u **koraku 2**. Pohlepnom odlukom za B ($d=7$) umesto D ($d=8$) primorao je sebe da plati skupu cenu povratka $D \rightarrow A$ ($d=10$). To je klasičan "**problem poslednje milje**" — kratkovidost ranih odluka kreira skupe ostanke za kraj. Na velikim grafovima ($n=100+$), NN heuristika u proseku daje rezultate **20-25% lošije** od optimuma, ali ima nisku složenost **$O(n^2)$** i trivijalne zahteve za memorijom.

6.2 Analiza ponašanja H2 (Greedy Edge Insertion)

H2 je uspešno izbegao lokalne zamke globalnim pogledom na problem. Ključan momenat je odbacivanje ivice B-C ($d=7$): iako jeftina, zatvorila bi izolovan krug A-C-B-A. Naredna ivica C-D ($d=8$) savršeno je spojila dve komponente i otvorila put ka optimalnom rešenju. Union-Find struktura omogućava proveru ciklusa u **$O(\alpha(n)) \approx O(1)$** , a ukupna složenost iznosi **$O(n^2 \log n)$** (dominira sortiranje ivica).

7. Implementacija u Python 3

Kompletan kod u fajlu **tspsolution.py** organizovan je modularno:

Funkcija	Opis	Složenost
<code>cena_rute(ruta)</code>	Izračunava ukupnu cenu date rute	$O(n)$
<code>brute_force(start)</code>	Iscrpna pretraga svih permutacija	$O(n!)$
<code>heuristika_nearest_neighbor(start)</code>	H1: Nearest Neighbor heuristika	$O(n^2)$
<code>heuristika_greedy_edge(start)</code>	H2: Greedy Edge + Union-Find	$O(n^2 \log n)$
<code>ispisi_rutu(naziv, ruta, cena)</code>	Formatiran ispis rezultata	$O(n)$

Tabela 7.1: Struktura Python implementacije

Stvarni izlaz programa (rezultat izvršavanja)

```
=====
PROBLEM TRGOVACKOG PUTNIKA - TSP (5 gradova)
Vestacka inteligencija, 2025/2026
=====

[GRAF - Matrica rastojanja]
      A      B      C      D      E
A  inf      7      6     10     13
B   7      inf      7    inf     10
C   6       7    inf      8      9
D  10     inf      8    inf      6
E  13     10      9      6    inf

=====

OPTIMALNO RESENJE (Brute Force)
Ruta : A -> B -> E -> D -> C -> A
Cena : 37

=====

H1: Nearest Neighbor
Ruta : A -> C -> B -> E -> D -> A
Cena : 39

Korak 1: Iz A -> kandidati: {B:7, C:6, D:10, E:13}
          Izabran: C (h=6)
Korak 2: Iz C -> kandidati: {B:7, D:8, E:9}
          Izabran: B (h=7)
Korak 3: Iz B -> kandidati: {D:inf, E:10}
          Izabran: E (h=10)
Korak 4: Iz E -> kandidati: {D:6}
          Izabran: D (h=6)

=====

H2: Greedy Edge Insertion
Ruta : A -> C -> D -> E -> B -> A
Cena : 37

Ivica A-C (d=6): ODABRANA
```

```
Ivica D-E (d=6): ODABRANA
Ivica A-B (d=7): ODABRANA
Ivica B-C (d=7): Formira preuranjeni krug
Ivica C-D (d=8): ODABRANA
Ivica C-E (d=9): Cvor C vec ima stepen 2
Ivica A-D (d=10): Cvor A vec ima stepen 2
Ivica B-E (d=10): ODABRANA
```

```
=====
Optimalno (BF): A->B->E->D->C->A   cena = 37
H1 (NN):        A->C->B->E->D->A   cena = 39
H2 (GE):        A->C->D->E->B->A   cena = 37
-> H2 pronalazi OPTIMALNO resenje!
=====
```

8. Implementacija u MATLAB / GNU Octave

Fajl **tsp.m** je funkcionalno ekvivalentan Python verziji, ali koristi MATLAB-specifičnu sintaksu i pristupe.

Aspekt	Python (tspsolution.py)	MATLAB (tsp.m)
Reprezentacija grafa	Dict rečnik {'A':{'B':7,...}}	Matrica DIST (n×n)
Gradovi	Lista stringova ['A','B',...]	Cell array {'A','B',...}
Permutacije	itertools.permutations()	perms() built-in
Union-Find	Python dict + while petlja	Ugnjezdene funkcije
Vizualizacija	Bez grafike (text output)	Grafički prikaz (figure/plot)
Ispis	print() / f-string	fprintf()
Složenost koda	~110 linija	~200 linija (grafika)

Tabela 8.1: Razlike između Python i MATLAB implementacije

Isečak MATLAB koda — Nearest Neighbor

```
function [route, cost, steps] = nearestNeighbor(DIST, start)
    n = size(DIST, 1);
    visited = false(1, n);
    route = start;
    visited(start) = true;
    current = start;
    steps = struct('from',{}, 'to',{}, 'dist',{});

    for i = 1:n-1
        best = inf; next = -1;
        for j = 1:n
            if ~visited(j) && DIST(current,j) < best
                best = DIST(current,j); next = j;
            end
        end
        steps(end+1) = struct('from',num2str(current), 'to',num2str(next), 'dist',best);
        route(end+1) = next; visited(next) = true; current = next;
    end
    route(end+1) = start; % Zatvaramo ciklus
    cost = calc(route, DIST);
end
```

Isečak MATLAB koda — Union-Find u Greedy Edge

```
% Union-Find ugnjezdene funkcije unutar greedyEdge
function p = find(x)
    while parent(x) ~= x
        parent(x) = parent(parent(x)); % Path compression
        x = parent(x);
    end
    p = x;
end

% Uslov prihvatanja ivice:
```

```
if degree(a)<2 && degree(b)<2 && ...  
    ~(find(a)==find(b) && size(chosen,1) degree(a)=degree(a)+1; degree(b)=degree(b)+1;  
    union(a,b);  
    chosen=[chosen; a b]; status='ODABRANA';  
end
```

MATLAB implementacija dodatno uključuje **grafički prikaz** sva tri puta istovremeno (optimum zelenom, H1 plavom isprekidanom, H2 magenta linijom) na koordinatnom sistemu sa ručno postavljenim pozicijama čvorova.

9. Zaključak i teorijska razmatranja

Na testiranoj instanci sa 5 gradova, eksperiment je dao jasne rezultate. **H2 (Greedy Edge Insertion)** pronašla je tačan globalni optimum (cena 37), dok je **H1 (Nearest Neighbor)** završio suboptimalno sa cenom 39 (+5.4%).

Kriterijum	H1: Nearest Neighbor	H2: Greedy Edge
Pronađena cena	39	37 (optimum)
Kvalitet rešenja	Suboptimalno (+5.4%)	Globalni optimum
Perspektiva algoritma	Lokalna (1 ■vor)	Globalna (ceo graf)
Složenost	$O(n^2)$	$O(n^2 \log n)$
Union-Find	Ne koristi	$O(\alpha(n)) \approx O(1)$
Skalabilnost	Lošija (~20–25% od opt.)	Bolja (manji prosek greške)
Lakoća implementacije	Trivijalna	Umerena

Tabela 9.1: Finalna komparativna analiza H1 vs H2

Teorijski zaključci

- H2 je robusnija i stabilnija heuristika** za TSP. Globalni pogled na ivice drastično smanjuje podložnost lokalnim minimumima i anomalijama rasporeda.
- H1 je opravdan** jedino kod izrazito malih grafova ($n \leq 10$) ili u sistemima sa kritičnim ograničenjima memorije gde se zahteva najjednostavnija logika pretrage.
- Odsustvo ivice B-D** pokazuje važnost analize strukture grafa pre primene algoritama — nepotpunost grafa direktno eliminišu određene Hamiltonove cikluse.
- Logičan nastavak** bio bi nadogradnja H2 algoritmima lokalnog poboljšanja (*2-opt*, *3-opt*) ili prelazak na metaheuristike — genetske algoritme ili simulirano kaljenje (*Simulated Annealing*) za instance sa $n > 1000$.

Konačni zaključak: Oba algoritma su pohlepna i nijedan ne garantuje optimalnost. Međutim, H2 je arhitekturno superiorno rešenje — globalni pristup u kombinaciji sa Union-Find strukturom daje konzistentno bolje rezultate uz prihvatljivu složenost $O(n^2 \log n)$.